三、LCEL高阶

1.Runnable协议核心函数

1.1 Runnable协议回顾

在之前的章节中，我们进行了runnable协议的基础内容讲解，并且使用管道符 "|"实现了模型模块的拼接，现在我们进行简单的回顾。

1.1.1 Runnable 协议概述

（1）**什么是 Runnable?**

Runnable 是 LangChain 中所有可执行组件遵循的**统一接口规范**（抽象基类）。

只要组件实现了 Runnable 接口，就可以使用统一的方式被调用和组合。LangChain 中几乎所有核心组件都实现了 Runnable，包括：

- 语言模型（ChatOpenAI 等）
- 提示模板（ChatPromptTemplate 等）
- 输出解析器（StrOutputParser 等）
- 检索器（Retriever 等）
- 自定义函数和链

（2）**Runnable 的核心价值**：

提供统一的调用方式和组合能力，让不同组件可以像标准零件一样灵活拼接。

发展历程简述：

- 早期：通过继承 `Chain` 类构建流程（组合性差、代码侵入性强）
- 现在：引入 Runnable 协议 + LCEL，使用声明式方式组合组件（更灵活、更简洁）

1.1.2 Runnable 的核心结构

（1）**核心设计理念**：

- **统一接口**：所有 Runnable 组件都支持相同的方法
- **声明式组合**：使用管道符 `|` 进行组合，像搭乐高一样直观
- **数据流抽象**：自动处理输入输出转换，开发者只需关注业务逻辑

（2）**核心方法**：

方法	类型	作用	常用场景
`invoke`	同步	单次执行	普通对话、单次推理
`stream`	同步	流式逐步返回结果	实时对话、长文本生成

方法	类型	作用	常用场景
`batch`	同步	批量执行	批量处理
`ainvoke`	异步	异步单次执行	高并发场景
`astream`	异步	异步流式输出	异步流式对话

教学重点：`invoke` 是基础，`stream` 是现代 LLM 应用的重要能力。

1.1.3 Runnable 的输入与输出

LangChain 通过 Runnable 对组件的输入输出进行了标准化处理。

（1）**常见组件输入输出**：

组件	主要输入	主要输出	说明
ChatOpenAI	str / PromptValue	AIMessage	核心生成组件
ChatPromptTemplate	dict	PromptValue	格式化提示
StrOutputParser	AIMessage	str	将模型输出转为字符串
RunnableLambda	Any	Any	自定义处理逻辑

（2）**关键类型说明**：

- **PromptValue**：提示模板与语言模型之间的桥梁

  提供 `to_string()` 和 `to_messages()` 两种转换方式。
- **AIMessage**：语言模型返回的主要结果对象

  常用属性：`content`（主要内容）、`response_metadata`、`usage_metadata`。
- **Output Parser 的作用**：

  把 AIMessage 转换为更干净、统一的输出格式（如字符串、JSON、结构化对象）。

1.1.4 LCEL（LangChain Expression Language）

（1）**什么是 LCEL?**

LCEL 是 LangChain 提供的**声明式组合语言**，核心是使用管道操作符 `|` 来连接多个 Runnable 组件。

经典写法：

```
chain = prompt | llm | parser
result = chain.invoke({"concept": "量子纠缠"})
```

（2）**LCEL 的优势**：

- 代码简洁直观

- 可读性强（从左到右清晰展示数据流）
- 自动处理类型转换
- 原生支持流式、并行、回退等高级能力

与传统 Chain 的对比：

- 传统方式：需要手动拼接，代码繁琐
- LCEL 方式：声明式组合，像“组装流程”而非“写代码”

（3）**简单回顾总结**：

Runnable 是 LangChain 实现组件统一调用和组合的基础协议，而 **LCEL** 是基于 Runnable 的声明式组合语言。掌握 `invoke`、`stream`、PromptValue、AIMessage 以及 `prompt | llm | parser` 的基本用法，是学习更高级 Runnable（Parallel、Assign 等）的前提。

1.2 Runnable 组件分类

在 Runnable 协议的基础上，LangChain 将可执行组件分为两类：**基础组件**和**高阶组件**。这种分类有助于我们理解不同组件在构建 LLM 应用时的定位和作用。

1.2.1 基础组件

（1）**什么是基础组件?**

基础组件是指 LangChain 中最核心、最常用的可执行单元，它们是构成 LCEL 流程的**原子级部件**。这些组件各自承担单一、明确的功能，可以独立使用，也可以被高阶组件组合使用。

简单来说，基础组件就像是搭建房屋时的砖块、水泥和钢筋，虽然单个组件功能有限，但它们是构建任何复杂 LLM 应用最基础的材料。没有这些基础组件，就无法组成完整的链和任务流。

（2）**基础组件的核心特点**：

- **功能单一且职责明确**：每个基础组件通常只负责一个相对独立的任务，不会把多种复杂逻辑混在一起。例如：
    - ChatPromptTemplate 只负责提示词的构建，不负责调用模型；
    - StrOutputParser 只负责输出格式转换，不参与内容生成。 这种单一职责的设计，使得组件更容易理解、测试和复用。
- **是 LCEL 管道中最常出现的“原子”组件**：在绝大多数 LCEL 链中，基础组件会以最高频率出现。几乎所有链都离不开 Prompt、LLM 和 Parser 的组合。它们是 LCEL 的“默认零件”，开发者在日常开发中接触最多的就是这一类组件。
- **通常不需要再进一步拆分**：基础组件的功能已经相对完整和原子化，一般不需要开发者再对其进行二次拆解。例如，我们通常不会去拆分 ChatPromptTemplate 的内部实现，而是直接使用它提供的 invoke 方法。

（3）**基础组件主要可以实现以下功能**：

功能类别	具体作用	代表组件	典型使用场景
输入构建	将用户输入格式化为模型可理解的提示	ChatPromptTemplate	几乎所有链

功能类别	具体作用	代表组件	典型使用场景
内容生成	调用大模型进行推理和生成	ChatOpenAI、ChatAnthropic 等	对话、问答、内容生成
输出处理	将模型输出转换为结构化或可读形式	StrOutputParser、JsonOutputParser	结构化输出、数据提取
知识检索	从外部知识库中检索相关信息	VectorStoreRetriever	RAG 系统
动作执行	执行具体工具或外部操作	Tool（@tool 装饰器包装后的函数）	Agent、工具调用
自定义逻辑	执行开发者自定义的处理逻辑	RunnableLambda	数据转换、后处理、简单判断

基础组件解决的是"**单一环节做什么**"的问题。它们是 LCEL 的基石，掌握好这些组件是学习高阶组件的前提。

1.2.2 高阶组件

（1）**什么是高阶组件?**

高阶组件是指那些**用于组合、控制和增强**基础组件的 Runnable。它们本身通常不直接生成内容或检索知识，而是通过特定的机制来组织多个基础组件的执行方式，从而实现更复杂的流程控制。

可以把高阶组件理解为搭建房屋时的结构框架和连接件。如果说基础组件是砖块和水泥，那么高阶组件就是决定这些砖块应该怎么排列、是否需要并排施工、是否要根据不同情况走不同路线的重要结构部件。

（2）**高阶组件的核心特点**：

- **主要用于流程控制和组合，而非直接执行任务**：高阶组件本身一般不产生最终结果，而是负责协调多个基础组件的执行方式。
    - 例如 RunnableParallel 不会自己生成答案，它只是同时运行多个组件并把结果收集起来；
    - RunnableAssign 也不会调用模型，它只是把某个组件的输出动态赋值给变量，供后续组件使用。
- **让 LCEL 具备更强的表达能力和灵活性**：基础组件只能完成单一功能，而高阶组件让开发者能够描述更复杂的执行逻辑，例如并行处理、动态注入上下文、条件判断等。这使得 LCEL 从简单的线性链，进化到可以表达更复杂业务流程的工具。
- **是实现复杂任务流的关键**：在实际项目中，当我们需要构建多步骤、多分支、需要并行处理的流程时，必须依赖高阶组件。例如：
    - 构建一个研究型 Agent，需要并行调用多个工具并动态组合上下文；
    - 构建合同审查流程，需要根据条款风险等级走不同审查路径。 这些复杂场景都离不开高阶组件的支撑。
- **通常需要与基础组件配合使用**：高阶组件很少单独使用，它们的作用是组织和控制基础组件。

可以说，**基础组件提供能力，高阶组件提供组织方式**，两者结合才能构建出真正有实用价值的复杂应用。

（3）**高阶组件主要可以实现以下功能**：

功能类别	具体作用	代表组件	解决的问题	典型应用场景
并行执行	同时运行多个 Runnable	RunnableParallel	提升效率、获取多路信息	多路检索、并行工具调用
动态上下文注入	将某个 Runnable 的输出动态赋值给变量	RunnableAssign	构建复杂、动态的 Prompt	高级 RAG、Agent 上下文管理
条件分支	根据输入条件选择不同的执行路径	RunnableBranch	实现简单路由逻辑	按问题类型走不同处理流程
数据透传	不做处理，直接将输入传递下去	RunnablePassthrough	保留原始输入供后续使用	配合 Assign 构建复杂输入
容错与回退	主流程失败时自动切换备用方案	with_fallbacks()	提升系统可靠性	生产环境部署

高阶组件解决的是"**多个组件应该怎么一起工作**"的问题。它们让开发者能够控制执行流程、提升性能、实现更复杂的业务逻辑，是从简单链进阶到生产级应用的关键能力。

1.3 基础组件：RunnableLambda

1.3.1 概述

RunnableLambda 是 LangChain 提供的一种轻量级工具，用于将**普通的 Python 函数**快速包装成符合 Runnable 协议的组件。它是 LCEL 中实现自定义逻辑最常用、最便捷的方式之一。

（1）为什么需要 RunnableLambda?

在构建 LLM 应用时，我们经常需要在链的某个位置加入一些**自定义处理逻辑**。比如：

- 对模型的输出进行格式化或补充说明；
- 根据输入内容做一些简单的判断或转换；
- 把多个检索结果拼接成一段清晰的上下文。

如果直接在链的外面写这些逻辑，代码就会变得零散，破坏 LCEL 原本简洁的声明式风格，也不利于后续维护。

RunnableLambda 就是为了解决这个问题而出现的。它允许我们把普通的 Python 函数包装成标准的 Runnable 组件，这样自定义逻辑就可以像 Prompt、LLM、Parser 一样，通过管道符自然地接入整个流程中。

概括来说，`RunnableLambda` 是连接"框架标准组件"和"开发者自定义逻辑"的桥梁，让 LCEL 既保持声明式的简洁，又具备足够的灵活性。

（2）RunnableLambda 的核心价值

价值点	说明	实际意义
易用性	只需一个普通函数即可使用，无需继承类	降低自定义逻辑的门槛
组合性	可以与其他 Runnable 组件自由组合	支持构建复杂任务流

价值点	说明	实际意义
一致性	包装后具备 `invoke`、`stream`、`batch` 等标准方法	保持 LCEL 风格统一
轻量级	适合简单、确定性的逻辑	避免过度设计
可扩展性	支持同步和异步函数	适应不同性能场景

1.3.2 基本使用示例

（1）封装自定义函数

```python
from langchain_core.runnables import RunnableLambda

# 定义一个函数：输入数字，返回加 6 后的结果
def add_six(number: int) -> int:
    return number + 6

# 使用 RunnableLambda 包装成 Runnable
add_six_runnable = RunnableLambda(add_six) # 注意函数作为参数传递时，不能带（）

# 调用
result = add_six_runnable.invoke(10)
print(result)   # 输出：16
```

（2）封装lambda函数

```python
from langchain_core.runnables import RunnableLambda

# 使用 lambda 表达式直接定义
add_six = RunnableLambda(lambda x: x + 6)

result = add_six.invoke(10)
print(result)   # 输出：16
```

（3）如何选择：用普通函数还是 lambda?

在实际使用 `RunnableLambda` 时，到底应该用普通函数（`def`）还是 `lambda` 表达式，主要取决于**逻辑复杂度**和**可读性**。下面给出实用建议：

- **大多数情况下，推荐使用普通函数（`def`）**，虽然 `lambda` 看起来更简洁，但当逻辑稍微复杂一点时，代码可读性会迅速下降。**可读性 > 简洁性** 是更重要的原则。
- **只有在逻辑非常简单时，才使用 `lambda`**，例如：加减法、拼接字符串、返回固定值等。

1.3.3 LCEL链中实现

（1）简单示例

```python
from langchain_core.runnables import RunnableLambda

sequence = RunnableLambda(lambda x:x-1) | RunnableLambda(lambda x: x * 2)
print(sequence.invoke(1))
```

（2）完整流程

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

# 1. 定义 Prompt
prompt = ChatPromptTemplate.from_template("请用一句话介绍 {topic}")

# 2. 定义模型
llm = ChatOpenAI(model="deepseek-chat")

# 3. 定义输出解析器
parser = StrOutputParser()

# 4. 定义 RunnableLambda：解析后拼接固定内容
def add_love(text: str) -> str:
    return text + "\n\n我爱大语言模型"

add_love_runnable = RunnableLambda(add_love)

# 5. 使用 LCEL 组合成完整链
chain = prompt | llm | parser | add_love_runnable

# 6. 调用
result = chain.invoke({"topic": "量子纠缠"})
print(result)
```

2.高阶 Runnable 组件

在掌握了基础组件和 RunnableLambda 之后，我们开始学习 高阶 Runnable 组件。这类组件的核心作用不是直接完成某个具体任务，而是组织和控制多个组件的执行方式。

高阶组件让 LCEL 具备了更强的流程控制能力，例如并行执行、动态上下文注入、条件分支等。通过学习高阶组件，我们可以构建出更复杂、更灵活的 LLM 任务流，这也是从简单链进阶到生产级应用的重要一步。

2.1 RunnablePassthrough + RunnableAssign（弱爆兄弟）

这对兄弟在 LCEL 里属于“有就行，没也行”的类型。严格来说，它俩的实用价值不算高，但既然官方有，我们还是简单了解一下，免得以后看到代码一脸懵。

2.1.1 RunnablePassthrough

`RunnablePassthrough` 是 LangChain 提供的一个高阶 Runnable 组件，它的作用非常简单：**不做任何处理，直接将输入原样传递下去。**

它可以理解为 LCEL 中的“直通管道”或“占位符”。理论上，当我们需要在链的某个位置保留原始输入或中间数据时，它会比较有用……理论上。

（1）为什么需要 RunnablePassthrough?

在构建复杂的 LCEL 链时，我们有时会希望把当前的数据保留下来，供后面的步骤继续使用。如果没有 RunnablePassthrough，我们就需要手动写代码来保留这些数据。

概括来说，RunnablePassthrough 主要用于在复杂流程中保留和传递数据，是配合 RunnableAssign 使用的一个辅助组件（虽然实际帮助有限）

（2）示例

```python
from langchain_core.runnables import RunnableLambda,RunnablePassthrough

sequence = (RunnableLambda(lambda x:X + 1)
           | RunnablePassthrough()        # 认真干活.jpg
           | RunnableLambda(lambda x: x * 2)
)
print(sequence.invoke(4))
# 输出：10
# 是的，RunnablePassthrough() 真的啥都没干
```

2.1.2 RunnableAssign

RunnablePassthrough 单独使用时作用有限，而它和 RunnableAssign 搭配使用……作用依然有限（乐）。

（1）作用

RunnablePassthrough + RunnableAssign 的主要作用是：

- 通过 RunnablePassthrough 保留原始输入数据（通常是字典）；
- 通过 RunnableAssign 在字典的基础上动态添加新的键值对（字段）。

注意：RunnableAssign 主要用于处理字典类型的数据，它会将新计算的字段合并到原字典中返回。

（2）如何配合

- RunnablePassthrough() 放在前面，负责把当前的数据原样传递下去。
- RunnableAssign() 放在后面，负责根据当前数据计算出新的字段，并将新字段合并到原有数据中。
- 两者结合后，链的后续步骤就可以同时使用原始数据和新添加的字段。

（3）示例

```python
from langchain_core.runnables import RunnablePassthrough, RunnableAssign

def add_greeting(data):
    return f"你好，{data['name']}！"

chain = (
    RunnablePassthrough()
    | RunnableAssign(greeting=add_greeting)
)

result = chain.invoke({"name": "李雷", "age": 18})
print(result)
```

输出：

```
{'name': '李雷', 'age': 18, 'greeting': '你好，李雷！'}
```

小结：

RunnablePassthrough 的主要功能就是“啥也不干，把数据原样传下去”，这活用 RunnableLambda 随便写两行就能干。RunnableAssign 虽然能比较优雅地给字典加字段，但本质上也逃不过被 RunnableLambda 替代的命运。

更扎心的是，**RunnableAssign 完全可以单干**，根本不需要 RunnablePassthrough 在旁边站桩。两者强行组 CP 之后，带来的提升其实也没多明显，更多时候只是让代码看起来“比较规范”而已。

所以结论就是：这对兄弟能用，但确实不强。属于那种“有它不多，无它不少”的存在。真正想好好处理复杂数据流和状态的时候，还是老老实实等 LangGraph 吧，这俩就别太当回事了。

2.2 RunnableParallel

2.2.1 概述

`RunnableParallel` 是 LangChain 中一个非常实用的高阶 Runnable 组件，它允许我们**并行执行多个 Runnable**，并将它们的执行结果收集到一个字典中返回。

简单来说，它的作用就是把多个任务“同时跑”，而不是一个接一个地串行执行。这在需要同时获取多种信息、调用多个工具，或者生成多个结果的场景中非常有用。

（1）为什么需要 RunnableParallel?

在构建 LLM 应用时，我们经常会遇到需要同时进行多个操作的情况，例如：

- 同时进行向量检索和关键词检索，以提高召回效果；
- 同时调用多个工具（Tool）来获取不同维度的信息；
- 对同一个问题从不同角度进行分析和回答。

如果使用普通的串行方式，这些操作会一个接一个执行，整体耗时较长。而 `RunnableParallel` 可以让这些任务**并发执行**，从而显著提升效率。

（2）RunnableParallel 的核心价值

- 就是**把多个 Runnable 放在一起并行运行**：它允许我们同时执行多个不同的操作，而不是一个接一个地串行执行。这样做可以明显提升整体执行效率，减少等待时间。
- RunnableParallel 会把每个 Runnable 的执行结果汇总成一个字典返回，方便我们在后续步骤中统一使用这些结果。

简单来说，它就是用来实现“多个任务一起跑”的工具，在需要并行处理多个子任务的场景中比较实用。

2.2.2 实现

（1）简单示例：显式写法

```
from langchain_core.runnables import RunnableParallel, RunnableLambda

# 使用 RunnableParallel 并行执行
parallel_chain = RunnableParallel({
    "sub_1": RunnableLambda(lambda x: x - 1),
    "sub_2": RunnableLambda(lambda x: x - 2)
})

result = parallel_chain.invoke(5)
print(result)
```

现实结果：

```
{'sub_1': 4, 'sub_2': 3}
```

（2）直接使用字典：隐式写法

```
from langchain_core.runnables import RunnableLambda

chain = RunnableLambda(lambda x: x ** 2)
        | {# type: ignore
            "sub_1": RunnableLambda(lambda x: x - 1),
            "sub_2": RunnableLambda(lambda x: x - 2)
}

print(chain.invoke(5))
```

这里当你把一个字典放在管道符"|"的右边时，LangChain 会自动把它转换成 RunnableParallel，不需要你手动写 RunnableParallel({...})。

注意：如果没有管道符，就会出错，所以使用的时候需要谨慎。

2.3 RunnableBranch

2.3.1 概述

RunnableBranch 是 LangChain 提供的一个高阶 Runnable 组件，用于实现**条件分支**逻辑。它可以根据输入的内容，选择执行不同的 Runnable。

简单来说，RunnableBranch 就像一个“条件判断器”，它会按照你预先定义的条件顺序，依次判断输入是否满足某个条件，如果满足就执行对应的 Runnable，否则继续判断下一个条件。

（1）为什么需要 RunnableBranch?

在构建 LLM 应用时，我们经常需要根据不同的输入走不同的处理逻辑，例如：

- 根据用户问题的类型（查询类、操作类、闲聊类），选择不同的处理流程；
- 根据输入内容的长度或复杂度，决定是直接回答还是先进行检索增强；
- 根据某些字段的值，选择调用不同的工具或模型。

如果使用普通的串行链，这些条件判断逻辑会写得很分散。而使用 RunnableBranch 可以把条件判断和分支逻辑集中管理，让代码结构更清晰。

（2）RunnableBranch 的核心特点

- 按照顺序依次匹配条件，第一个匹配成功的条件对应的 Runnable 会被执行；
- 必须提供一个默认的 Runnable（通常放在最后），当所有条件都不满足时执行默认分支；
- 输入可以是任意类型，条件判断函数需要返回布尔值（True 或 False）；
- 适合处理“多路分支”的场景，是实现简单路由逻辑的常用方式。

2.3.2 结构

`RunnableBranch` 的初始化结构如下：

```
RunnableBranch(
    (condition1, runnable1),
    (condition2, runnable2),
    ...
    default_runnable    # 最后一个参数是默认分支（必须提供）
)
```

结构组成：

- 它接收多个**元组** `(condition, runnable)`。
- 最后一个参数**不是元组**，而是直接传入一个 Runnable，作为**默认分支。**
- 每个 `condition` 是一个**可调用对象**（函数或 lambda），接收输入并返回 `True` 或 `False`。

2.3.3 内部执行逻辑

（1）`RunnableBranch` 的执行流程

1. 接收输入（`input`）。
2. **按顺序**遍历所有条件分支。
3. 对每个 `(condition, runnable)`：
    - 调用 `condition(input)`。
    - 如果返回 `True`，则执行对应的 `runnable(input)`，并返回结果。
4. 如果所有条件都不满足，则执行**默认分支**（最后一个参数）。
5. 返回最终结果。

执行顺序非常重要：条件是**从上到下**依次判断的，一旦有一个条件满足，后面的条件就不会再判断。

（2）与 if-elif-else 的对比

RunnableBranch 的执行流程和 Python 中的 if-elif-else 结构**高度相似**：

方面	if-elif-else	RunnableBranch	相似度
判断方式	从上到下依次判断条件	从上到下依次判断条件	非常相似
执行逻辑	第一个满足条件的代码块被执行	第一个满足条件的 Runnable 被执行	非常相似
短路机制	一旦条件满足，后续不再判断	一旦条件满足，后续不再判断	非常相似
默认分支	else	最后一个参数（默认 Runnable）	非常相似
表达形式	命令式（写代码块）	声明式（组合 Runnable 组件）	不同
可组合性	较弱	很强（可以和其他 Runnable 自由组合）	不同

结论： RunnableBranch 的**逻辑结构**和 if-elif-else 几乎一模一样，只是把“执行代码块”换成了“执行 Runnable 组件”。

2.3.4 核心结构特点

组成部分	类型	作用	是否必须
条件函数	Callable	判断输入是否满足该分支	是
分支 Runnable	Runnable	满足条件时执行的处理逻辑	是
默认分支	Runnable	所有条件都不满足时执行的兜底逻辑	是
执行顺序	列表顺序	决定条件判断的优先级	-

2.3.5 示例

```python
from langchain_core.runnables import RunnableBranch, RunnableLambda

# 使用 RunnableBranch + lambda 实现根据字符串长度进行不同处理
text_processor = RunnableBranch(
    # 长度 ≤ 20，直接返回原文
    (lambda x: len(x) <= 20, RunnableLambda(lambda x: x)),

    # 长度在 20~50 之间，返回前10个 + ... + 后10个
    (lambda x: 20 < len(x) <= 50, RunnableLambda(lambda x: x[:10] + "..." + x[-10:])),

    # 默认分支（长度 > 50），返回前20个 + ... + 后20个
    RunnableLambda(lambda x: x[:20] + "..." + x[-20:])
)

# 测试
texts = [
    "Hello World",                                          # 短文本（≤20）
    "This is a medium length text for testing.",            # 中等长度（20~50）
    "This is a very long text used to test RunnableBranch.", # 长文本（>50）
]

for text in texts:
    result = text_processor.invoke(text)
    print(f"原文长度: {len(text):<3} → {result}")
```

3. Workflow

3.1 Workflow概述

3.1.1 什么是 Workflow

（1）Workflow 的概念

假设我们要让模型回答一个问题，最基础的处理流程是这样的：

```
用户问题 -> 提示词构造（Prompt） -> 大模型生成答案（LLM） -> 结果输出（OutputParser）
```

用 LCEL 写出来大概是下面这样：

```
prompt | llm | parser
```

这个流程虽然简单，但它其实已经包含了多个步骤，并且这些步骤是按照一定顺序被组织起来，共同完成“回答用户问题”这个目标的。

当我们把多个步骤按照一定逻辑组织起来执行时，我们其实就是在构建一个 Workflow（工作流）。

（2）**Workflow 的本质**

Workflow 的核心，是对任务执行过程进行结构化编排，它关注的是：

- 多个任务应该以什么顺序执行
- 不同任务之间如何传递数据
- 在什么条件下执行哪个任务

简单来说，Workflow 解决的是“怎么把多个能力组织起来完成一个任务”这个问题，而不是某个单一节点（比如 LLM）本身有多强。

（3）举例说明

比如我们要处理一份 PDF 报告，如果直接让大模型“一口气分析完”，效果往往不稳定。但如果我们把它拆解成以下流程：

```
PDF 文档 -> 文本提取 -> 内容总结 -> 关键信息提取 -> 生成分析报告
```

这个过程就是一个 Workflow。它通过把复杂任务拆解成多个步骤，并规定这些步骤的执行顺序和协作方式，来提高整个流程的可靠性和可控性。

3.1.2 Workflow 与 Agent关系

（1）Agent 替换 LLM

前面的流程中，我们将 Prompt 直接发送给 LLM，由模型生成最终结果：

```
用户问题 -> 提示词构造（Prompt） -> LLM -> 结果输出
```

那么，如果把中间的 **LLM** 换成 **Agent** 呢?

```
用户问题 -> 提示词构造（Prompt） -> Agent -> 结果输出
```

从表面上看，好像只是替换了一个节点，但实际上这个节点的能力发生了变化。

- 传统 LLM 更像一个“执行者”，接收到输入后直接生成输出
- 而 Agent 除了具备语言生成能力之外，还拥有工具调用、任务规划、状态管理以及决策能力。

当 Agent 接收到任务时，它可以根据当前情况决定是否调用工具、是否需要获取外部信息，以及下一步应该执行什么操作。但需要特别注意的是：

Agent 并不等于整个系统，也不等于 Workflow。

（2）Agent 只是一个节点

Agent 本质上仍然只是整个执行流程中的一个节点，只不过这个节点比普通 LLM 更智能、更具自主决策能力。

例如下面这个流程：

```
用户问题 -> 意图识别 -> Agent -> 工具调用 -> 结果整理 -> 结果输出
```

这里真正定义整体执行过程的，仍然是 Workflow，因此可以这样理解：

```
Workflow = 整个流程的组织者
Agent = 流程中的智能决策节点
LLM = Agent 或其他节点背后的推理能力
```

引入 Agent 并不是为了替代 Workflow，而是为了让 Workflow 中的某些节点具备更强的自主决策能力。整个系统的核心仍然是任务如何拆分、如何协作以及如何组织执行流程，而这正是 Workflow 所解决的问题。

3.2 Workflow 的组成结构

3.2.1 节点（Node）

在 Workflow 中，**节点（Node）** 是最基本的执行单元。每一个节点都负责完成 Workflow 中的某一个具体任务或功能。

节点可以是各种不同类型的组件，例如：

- **Prompt**：负责构造发送给模型的提示词，是连接用户输入和模型之间的桥梁。
- **LLM（大模型）**：负责内容的理解、推理和生成，是 Workflow 中最核心的智能节点。
- **Tool（工具）**：让 Agent 或 Workflow 能够与外部世界交互，例如调用搜索引擎、执行代码、查询数据库等。
- **Retriever（检索器）**：负责从知识库中检索相关信息，是 RAG 类应用中的重要节点。
- **Python 函数**：通过 RunnableLambda 包装的自定义逻辑，用于数据处理、格式转换、条件判断等非模型相关的操作。

节点的核心意义在于：Workflow 并不是由单一类型的组件构成的，而是可以灵活组合不同能力的节点来完成复杂任务。每个节点都有明确的输入和输出，节点之间通过数据进行连接和协作。

3.2.2 数据流（Data Flow）

数据流 是指信息在 Workflow 各个节点之间传递的过程。

在 Workflow 中，数据并不是孤立存在的，而是会从一个节点流向下一个节点。每个节点接收上游节点传递过来的数据，经过处理后，再将结果传递给下游节点。

在这个过程中，数据不断被加工和传递：从原始问题变成检索查询，再变成上下文，最后变成模型的回答。

数据流的重要性在于：

- 它决定了信息如何在不同节点之间流动；
- 它影响最终输出的质量和准确性；
- 清晰的数据流设计能够让 Workflow 更易于理解、调试和维护。

3.2.3 控制流（Control Flow）

控制流 是指 Workflow 中决定任务执行路径的逻辑。它规定了各个节点应该按照什么方式被执行。

控制流主要包括以下几种类型：

- **顺序执行**：节点按照固定顺序依次执行，上一个节点执行完毕后，下一个节点才会开始。这是大多数基础

Workflow 的默认执行方式。

- **并行执行**：多个节点可以同时执行，而不需要等待前一个节点完成。常用于需要同时获取多种信息或执行多个独立任务的场景，例如同时进行向量检索和关键词检索。
- **条件执行**：根据当前数据的状态或内容，选择执行不同的后续节点。这对应我们之前学过的 RunnableBranch，用于实现“如果...就...否则...”的逻辑。
- **循环执行**：在满足特定条件之前，重复执行某些节点或子流程。虽然纯 LCEL 对循环的支持有限，但在更复杂的 Workflow 中（如 LangGraph），循环是非常常见和重要的控制方式。

控制流决定了 Workflow 的**执行逻辑和灵活性**。一个简单的 Workflow 可能只包含顺序执行，而一个复杂的 Workflow 则可能同时包含并行、条件甚至循环等多种控制方式。

3.3 从 Workflow 到 Agentic Workflow

前面我们已经把 Workflow 的本质讲清楚了：它是对任务执行过程的结构化编排，核心是把多个节点（Prompt、LLM、Tool、Retriever、Python 函数等）按照一定逻辑组织起来。

然而，当我们把 Workflow 应用到真正复杂的真实任务时，会发现传统 Workflow 存在明显的局限。

3.3.1 传统 Workflow 的问题

传统 Workflow 的最大特点是**流程固定**。开发者需要提前把整个流程设计好：先做什么、后做什么、什么时候分支、什么时候并行，都要写死在代码或可视化画布里。这种方式在处理**确定性强、步骤清晰**的任务时非常有效。

但面对复杂且充满不确定性的真实任务时，问题凸显：

- 中间结果可能不符合预期，需要回溯调整前面步骤
- 某个子任务发现数据缺失或假设错误，可能需要整个流程重新执行

示例：商业分析项目包含市场调研 → 竞品分析 → 用户画像 → 财务模型 → 结论输出。实际执行中经常出现“调研结果影响用户画像、模型缺失数据又需回溯竞品分析”的情况。固定流程难以支持这种“边做边调整”的动态需求。

核心局限：流程僵化，缺乏根据中间结果动态决策和迭代的能力。

3.3.2 Agentic Workflow

传统 Workflow 的核心问题是**流程固定、缺乏动态适应能力**。当任务复杂且充满不确定性时，中间结果往往需要回溯调整，而固定路径难以支持这种“边做边改”的需求。

Agentic Workflow 通过将 Agent 引入 Workflow，解决了这一局限。它让 Workflow 从“预定义的固定流水线”升级为“由智能节点驱动的动态流程”。

一个典型的 Agentic Workflow 大致是这样的：

```
任务下发
   ↓
Agent 进行规划（决定需要哪些步骤、顺序如何）
   ↓
执行具体任务（可能调用 Tool、Retriever、Python 函数等）
   ↓
Agent 对执行结果进行检查和反思
   ↓
如果不满足要求 → 返回上一步重新规划或执行
   ↓
输出最终结果
```

Agent 的角色定位（核心澄清）

在 Agentic Workflow 中，**Agent 不是只参与某一个环节，而是作为智能 Orchestrator（编排者）全程参与规划、执行决策和优化**：

- **规划（Planning）**：Agent 负责动态拆解任务、生成或调整执行计划，而非由开发者提前写死所有步骤。
- **执行决策（Execution）**：Agent 实时决定"下一步调用哪个节点/工具"，并通过调用驱动执行。具体的底层操作（如检索细节、代码运行）由被调用的专业节点完成，Agent 主要负责"做什么、何时做"的智能决策。
- **优化与反思（Optimization & Reflection）**：Agent 检查中间结果，判断是否满足要求，若不满足则主动发起迭代（重新规划、补充信息或调整路径）。

Agent 驱动了整个流程的智能决策与自适应控制，但它并不替代所有底层执行，而是与 Workflow 中预定义的节点（Tool、Retriever 等）协同工作。Workflow 的骨架（可用节点、状态管理、可能路径）仍由开发者设计，Agent 在这个框架内实现动态编排。

3.3.3 为什么会出现 Context Engineering?

即使我们已经把 Workflow 升级为 **Agentic Workflow**，让 Agent 负责动态规划、执行决策和反思迭代，实际效果仍然经常不稳定。同一任务、同一个 Agent，有时表现优秀，有时却严重跑偏或陷入循环。

根本原因在于：**Agent 的能力 = 模型能力 + 上下文质量**

> **上下文** 是指当前步骤中提供给模型或 Agent 的**全部输入信息集合**。 它不仅包括用户问题和系统提示词，还包括对话历史、检索到的知识、工具描述、Agent 之前的思考过程（thought）、工具执行结果（observation）、状态信息等。在 Agentic Workflow 中，上下文是动态变化的，会随着每一步执行不断累积、更新、过滤和迭代。

- 模型能力由所选大模型决定（相对固定）。
- 上下文质量则完全可以由我们人为控制和优化。

当上下文组织得混乱、关键信息缺失、结构不清、或重要信号被大量无关内容淹没时，再聪明的 Agent 也难以做出正确的规划、工具选择和反思决策。反之，如果上下文清晰、完整、结构化且信息密度高，Agent 的表现会显著提升。

传统做法的局限：

- 早期主要靠"写好 Prompt"来解决上下文问题（Prompt Engineering）。
- 但在 Agentic Workflow 中，上下文不再是静态的一次性输入，而是**动态流动、多次迭代、多节点协作**的

资源。单纯优化单个 Prompt 已经远远不够。

Context Engineering 正是在这个背景下出现的。 它把“上下文”当作一个需要被系统性工程化管理的核心资源，关注如何在 Workflow（尤其是 Agentic Workflow）中：

- 科学地**设计**上下文的结构与内容
- 高效地**组织**和**传递**上下文信息
- 动态地**管理**上下文的更新、过滤、压缩与迭代

简单来说： **Context Engineering 解决的是“即使有了强大的 Agent，上下文质量不足仍会导致系统不稳定”这一核心瓶颈。**它让 Agent 能够在高质量上下文的支持下，稳定地发挥规划、决策和优化能力。

一句话总结： 从 Workflow 到 Agentic Workflow，我们解决了“流程如何动态编排”的问题；而 Context Engineering 进一步解决了“动态流程中，Agent 到底能拿到什么质量的上下文”这一更深层的问题。它是 Agentic 系统走向稳定可用的关键工程实践。